

Отчет по лабораторной работе №5

Основы информационной безопасности

Шихалиева З.А НПИбд-03-23

Содержание

1	Цель работы	5
2	Теоретическое введение	6
3	Выполнение лабораторной работы	8
4	Выводы	18
	Список литературы	19

Список иллюстраций

3.1	Подготовка к лабораторной работе	8
3.2	Вход от имени пользователя guest	9
3.3	Создание файла	9
3.4	Содержимое файла	9
3.5	Компиляция файла	10
3.6	Сравнение команд	10
3.7	Создание и компиляция файла	10
3.8	Содержимое файла	11
3.9	Смена владельца файла и прав доступа к файлу	11
3.10	Запуск файла	12
3.11	Создание и компиляция файла	12
3.12	Содержимое файла	13
3.13	Смена владельца файла и прав доступа к файлу	13
3.14	Попытка прочесть содержимое файла	14
3.15	Попытка прочесть содержимое файла программой	14
3.16	Попытка прочесть содержимое файла программой	14
3.17	Чтение файла от имени суперпользователя	14
3.18	Проверка атрибутов директории tmp	15
3.19	Создание файла, изменение прав доступа	15
3.20	Попытка чтения файла	15
3.21	Попытка записи в файл	16
3.22	Попытка удалить файл	16
3.23	Смена атрибутов файла	16
3.24	Проверка атрибутов директории	16
3.25	Повтор предыдущих действий	17
3.26	Изменение атрибутов	17

Список таблиц

1 Цель работы

Изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

2 Теоретическое введение

1. Дополнительные атрибуты файлов Linux

В Linux существует три основных вида прав — право на чтение (read), запись (write) и выполнение (execute), а также три категории пользователей, к которым они могут применяться — владелец файла (user), группа владельца (group) и все остальные (others). Но, кроме прав чтения, выполнения и записи, есть еще три дополнительных атрибута. [1]

Sticky bit

Используется в основном для каталогов, чтобы защитить в них файлы. В такой каталог может писать любой пользователь. Но, из такой директории пользователь может удалить только те файлы, владельцем которых он является. Примером может служить директория /tmp, в которой запись открыта для всех пользователей, но нежелательно удаление чужих файлов.

SUID (Set User ID)

Атрибут исполняемого файла, позволяющий запустить его с правами владельца. В Linux приложение запускается с правами пользователя, запустившего указанное приложение. Это обеспечивает дополнительную безопасность т.к. процесс с правами пользователя не сможет получить доступ к важным системным файлам, которые принадлежат пользователю root.

SGID (Set Group ID)

Аналогичен suid, но относится к группе. Если установить sgid для каталога, то все файлы созданные в нем, при запуске будут принимать идентификатор группы каталога, а не группы владельца, который создал файл в этом каталоге.

Обозначение атрибутов `sticky`, `suid`, `sgid`

Специальные права используются довольно редко, поэтому при выводе программы `ls -l` символ, обозначающий указанные атрибуты, закрывает символ стандартных прав доступа.

Пример: `rwsrwsrwt`

где первая `s` — это `suid`, вторая `s` — это `sgid`, а последняя `t` — это `sticky bit`

В приведенном примере не понятно, `rwt` — это `rw-` или `gwx`? Определить это просто. Если `t` маленькое, значит `x` установлен. Если `T` большое, значит `x` не установлен. То же самое правило распространяется и на `s`.

В числовом эквиваленте данные атрибуты определяются первым символом при четырехзначном обозначении (который часто опускается при назначении прав), например в правах `1777` — символ `1` обозначает `sticky bit`. Остальные атрибуты имеют следующие числовое соответствие:

1 — установлен `sticky bit`

2 — установлен `sgid`

4 — установлен `suid`

2. Компилятор GCC

GCC - это свободно доступный оптимизирующий компилятор для языков C, C++. Собственно программа `gcc` это некоторая надстройка над группой компиляторов, которая способна анализировать имена файлов, передаваемые ей в качестве аргументов, и определять, какие действия необходимо выполнить. Файлы с расширением `.cc` или `.C` рассматриваются, как файлы на языке C++, файлы с расширением `.c` как программы на языке C, а файлы с расширением `.o` считаются объектными [2].

3 Выполнение лабораторной работы

Для лабораторной работы необходимо проверить, установлен ли компилятор gcc, команда `gcc -v` позволяет это сделать. Также осуществляется отключение системы запретов с помощью `setenforce 0` (рис. 1).

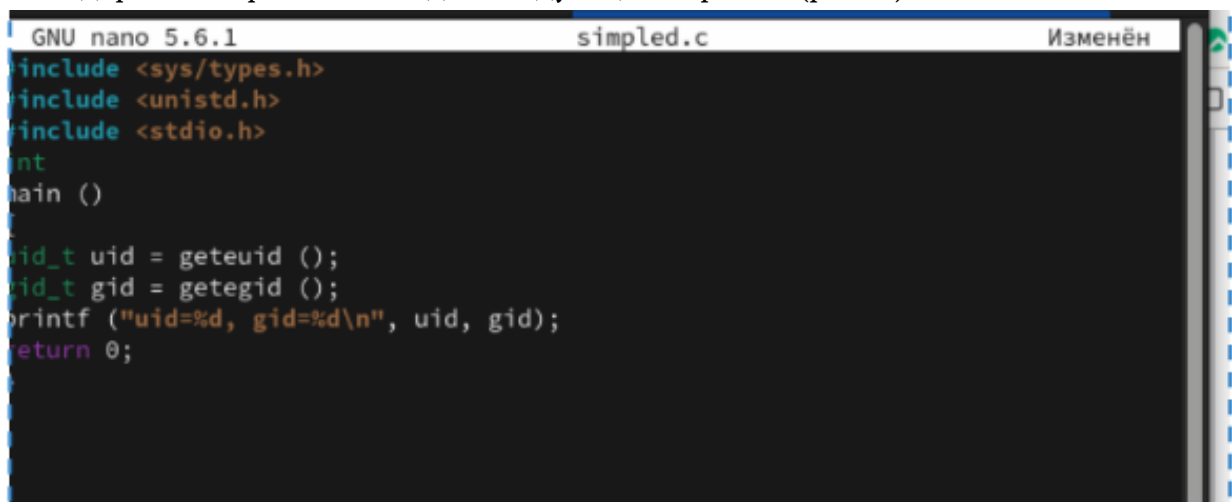
```
Используются внутренние спецификации.  
COLLECT_GCC=gcc  
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-redhat-linux/11/lto-wrapper  
OFFLOAD_TARGET_NAMES=nvptx-none  
OFFLOAD_TARGET_DEFAULT=1  
Целевая архитектура: x86_64-redhat-linux  
Параметры конфигурации: ../configure --enable-bootstrap --enable-host-pie --enable-host-bind-now --enable-languages=c,c++,fortran,lto --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bugurl=https://bugs.rockylinux.org/ --enable-shared --enable-threads=posix --enable-checking=release --with-system-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-gnu-unique-object --enable-linker-build-id --with-gcc-major-version-only --enable-plugin --enable-initfini-array --without-isl --enable-multilib --with-linker-hash-style=gnu --enable-offload-targets=nvptx-none --without-cuda-driver --enable-gnu-indirect-function --enable-cet --with-tune=generic --with-arch_64=x86-64-v2 --with-arch_32=x86-64 --build=x86_64-redhat-linux --with-build-config=bootstrap-lto --enable-link-serialization=1  
Модель многопоточности: posix  
Supported LTO compression algorithms: zlib zstd  
gcc версия 11.4.1 20230605 (Red Hat 11.4.1-2) (GCC)
```

Рис. 3.1: Подготовка к лабораторной работе

Осуществляется вход от имени пользователя `guest` (рис. 2).

C++ Листинг 1 `#include <sys/types.h> #include <unistd.h> #include <stdio.h> int main () { uid_t uid = geteuid (); gid_t gid = getegid (); printf ("uid=%d, gid=%d\n", uid, gid); return 0; }`

Содержимое файла выглядит следующим образом (рис. 4)



```
GNU nano 5.6.1 simplified.c Изменён
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t uid = geteuid ();
    gid_t gid = getegid ();
    printf ("uid=%d, gid=%d\n", uid, gid);
    return 0;
}
```

Рис. 3.4: Содержимое файла

Компилирую файл, проверяю, что он скомпилировался (рис. 5)

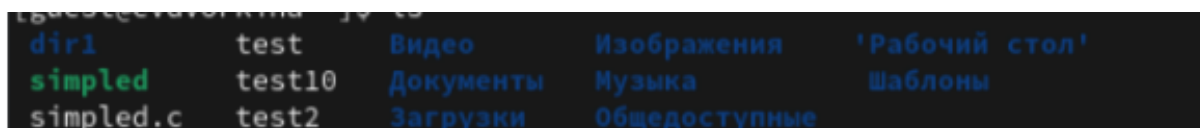


Рис. 3.5: Компиляция файла

Запускаю исполняемый файл. В выводе файла выписаны номера пользователя и групп, от вывода при вводе if, они отличаются только тем, что информации меньше (рис. 6)

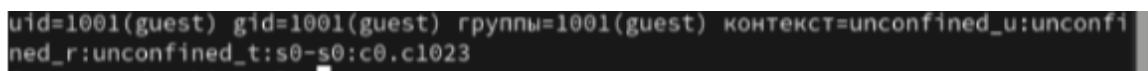


Рис. 3.6: Сравнение команд

Создание, запись в файл и компиляция файла simplified2.c. Запуск программы (рис. 7)

Рис. 3.7: Создание и компиляция файла

C++ Листинг 2 #include <sys/types.h> #include <unistd.h> #include <stdio.h> int main () { uid_t real_uid = getuid (); uid_t e_uid = geteuid (); gid_t real_gid = getgid (); gid_t e_gid = getegid ();

```
printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid); printf ("real_uid=%d,  
real_gid=%d\n", real_uid, real_gid); return 0; }
```

(рис. 8)



```
GNU nano 5.6.1 simplified2.c Изменён
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t real_uid = getuid ();
    uid_t e_uid = geteuid ();
    gid_t real_gid = getgid ();
    gid_t e_gid = getegid ();
    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid, real_gid);
    return 0;
}
```

Рис. 3.8: Содержимое файла

С помощью `chown` изменяю владельца файла на суперпользователя, с помощью `chmod` изменяю права доступа (рис. 9)

Рис. 3.9: Смена владельца файла и прав доступа к файлу

Сравнение вывода программы и команды `id`, наша команда снова вывела только ограниченное количество информации (рис. 10)



Рис. 3.11: Создание и компиляция файла

```
С++ Листинг 3 #include <fcntl.h> #include <stdio.h> #include  
<sys/stat.h> #include <sys/types.h> #include <unistd.h> int main (int  
argc, char* argv[]) { unsigned char buffer[16]; size_t bytes_read;  
int i; int fd = open (argv[1], O_RDONLY); do { bytes_read = read (fd,  
buffer, sizeof (buffer)); for (i =0; i < bytes_read; ++i) printf("%c",  
buffer[i]); } while (bytes_read == sizeof (buffer)); close (fd);  
return 0; }
```

(рис. 12)

```
GNU nano 5.6.1                                readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
}
```

Рис. 3.12: Содержимое файла

Снова от имени суперпользователя меняю владельца файла readfile. Далее меняю права доступа так, чтобы пользователь guest не смог прочесть содержимое файла (рис. 13)

```
sudo chown root:guest /home/guest/readfile.c
sudo chmod u+s /home/guest/readfile.c
sudo chmod 700 /home/guest/readfile.c
sudo chmod -r /home/guest/readfile.c
sudo chmod u+s /home/guest/readfile.c
```

Рис. 3.13: Смена владельца файла и прав доступа к файлу

Проверка прочесть файл от имени пользователя guest. Прочесть файл не удастся (рис. 14)

4 Выводы

Изучила механизм изменения идентификаторов, применила SetUID- и Sticky-биты. Получила практические навыки работы в консоли с дополнительными атрибутами. Рассмотрела работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

Список литературы

1. Дополнительные атрибуты файлов: sticky bit, suid, sgid [Электронный ресурс]. 2018. URL: <https://tokmakov.msk.ru/blog/item/141>.
2. Инструментарий программиста в Linux: Компилятор GCC [Электронный ресурс]. URL: <http://parallel.imm.uran.ru/freesoft/make/instrum.html>.